

11_ 贪心法

11 贪心法

题目原文

贪心法的思想和特点，用贪心法解决经典问题并能给出证明思路：换硬币、单源最短路径。

考查类型

概念题、算法设计题和证明题。

贪心法思想

贪心法在每一步都做当前看来最优的选择，并希望这些局部选择组合成全局最优解。
适用贪心法通常需要两个性质：

贪心选择性质
最优子结构

贪心选择性质表示存在一个全局最优解包含当前的局部最优选择。

最优子结构表示做出一次选择后，剩余问题的最优解可以和当前选择组合成原问题最优解。

证明思路

常用证明方法有两类。

交换论证：

从任意最优解出发，把它逐步替换成包含贪心选择的最优解，同时不降低解的质量。

领先性证明：

证明贪心算法在每一步都不差于任何最优解的对应前缀。

换硬币问题

目标是在给定币值系统下，用尽量少的硬币凑出金额 N 。

贪心策略：

每次选择不超过剩余金额的最大面值硬币

重要点：

不是所有币值系统都能用贪心得得到最优解。答题时必须结合题目给出的币值系统证明贪心选择性质。

证明结构：

1. 说明贪心第一步选取最大可用面值。
2. 证明存在一个最优解也包含这枚硬币。
3. 删除这枚硬币后，剩余金额构成同类子问题。
4. 对子问题重复使用同样论证。

若题目给出标准币值系统，需要用该系统的面值关系证明“大面值替换多个小面值不会增加硬币数”。

单源最短路径

单源最短路径的典型贪心算法是 Dijkstra 算法。它要求边权非负。

算法思想：

1. 维护源点到各点的当前最短估计 $\text{dist}[v]$ 。
2. 每次从未确定顶点中取 dist 最小的顶点 u 。
3. 将 u 标记为已确定。
4. 用 u 的出边松弛相邻顶点。

伪代码：

```
Dijkstra(G, s):  
  for each vertex v:  
     $\text{dist}[v] = \infty$   
   $\text{dist}[s] = 0$   
  
   $Q = \text{allvertices}$   
  
  while Q is not empty:  
     $u = \text{vertex in } Q \text{ with minimum } \text{dist}[u]$   
    remove u from Q  
    for each edge (u, v):  
      if  $\text{dist}[v] > \text{dist}[u] + w(u, v)$ :  
         $\text{dist}[v] = \text{dist}[u] + w(u, v)$ 
```

Dijkstra 正确性证明

关键命题：

每次取出的 dist 最小未确定顶点 u ，其 $\text{dist}[u]$ 已经等于源点到 u 的真实最短距离。

证明思路：

若存在一条更短路径从源点到 u ，考虑这条路径上第一个未确定顶点 y ，以及它前一个已确定顶点 x 。由于 x 已经确定，边 (x, y) 已经被松弛，因此 $\text{dist}[y]$ 不会大于该路径到 y 的长度。边权非负，所以到 y 的长度不大于到 u 的长度。这会与 u 是未确定顶点中 dist 最小者矛盾。

因此贪心选择正确。

例题

例题 1

题目：币值为：

1, 2, 4, 8, 16

用贪心法凑出金额 23，并说明最优性。

解答：

贪心选择：

$$23 = 16 + 4 + 2 + 1$$

共用 4 枚硬币。

这些币值都是 2 的幂。金额 23 的二进制表示为：

$$23 = 16 + 4 + 2 + 1$$

每个二进制位对应一枚硬币。使用更小面值替代其中任意一枚硬币，都不会减少硬币数。因此该贪心解是最优解。

例题 2

题目：对下面非负权图从源点 s 运行 Dijkstra 算法：

$$s \rightarrow a : 1$$

$$s \rightarrow b : 4$$

$$a \rightarrow b : 2$$

$$a \rightarrow c : 5$$

$$b \rightarrow c : 1$$

$$b \rightarrow d : 7$$

$$c \rightarrow d : 3$$

求从 s 到各点的最短距离。

解答：

初始化：

```
dist[s] = 0
dist[a] = ∞
dist[b] = ∞
dist[c] = ∞
dist[d] = ∞
```

从 s 松弛：

```
dist[a] = 1
dist[b] = 4
```

取出 a ，松弛：

```
dist[b] = min(4, 1 + 2) = 3
dist[c] = 1 + 5 = 6
```

取出 b ，松弛：

```
dist[c] = min(6, 3 + 1) = 4
dist[d] = 3 + 7 = 10
```

取出 c ，松弛：

```
dist[d] = min(10, 4 + 3) = 7
```

最终结果：

```
dist[s] = 0
dist[a] = 1
dist[b] = 3
dist[c] = 4
dist[d] = 7
```

常见误区

贪心算法必须证明正确性，不能只写“每一步取最优”。

Dijkstra 算法不能直接用于含负权边的图。

换硬币问题不能默认所有币值系统都适合贪心。